QATIP Intermediate AWS Lab04 Terraform Modules

Contents

Lab Objectives 2
Before you begin 2
Try It Yourself 2
Challenge 2
Key Requirements 2
Step-by-Step Instructions 3
Set Up the Project Directory 3
Create the VPC Module 3
Create the Root module Configuration 4
Initialize, Review and Apply 5
Verify the Deployment 5
Clean Up Resources 5
Demo Lab 6
Lab Logic 6
Stretch Challenge (Time permitting) ..... **Error! Bookmark not defined.**
Challenge Objectives ..... **Error! Bookmark not defined.**
Key Concepts Practiced ..... **Error! Bookmark not defined.**
Solution ..... **Error! Bookmark not defined.**
Lab Clean Up ..... **Error! Bookmark not defined.**

© QA 2026

Lab Objectives

1. Develop reusable Terraform modules for AWS VPCs.
2. Parameterize the module using variables.
3. Deploy multiple VPCs by reusing the module with parameters.
4. Utilize module outputs in the root module to create dependent resources.

Before you begin

5. Ensure you have completed Lab0 before attempting this lab.
6. In the IDE terminal pane, enter the following command...

   cd ~/environment/aws-tf-int/labs/04

7. This shifts your current working directory to labs/04. Ensure all commands are executed in this directory.
8. Close any open files and use the Explorer pane to navigate to and open the labs/04 folder. The provisioned content is for later use.

Try It Yourself

Challenge

9. Using **labs/04** as your root directory and ignoring the provisioned content, create a reusable module structure to deploy two AWS VPCs in the same AWS region, **us-west-2**. As you deploy the module, expose outputs from it and use these outputs in the root module to create a subnet on each VPC.

Key Requirements

- Create main.tf in aws-tf-int/labs/04
- Create main.tf, variables.tf, and outputs.tf files in aws-tf-int/labs/04/modules/vpc
- Deploy two virtual networks, vpc-01 with CIDR 10.0.0.0/16 and vpc-02 with CIDR 10.1.0.0/16, using a vpc module structure
- Pass output values from the module back to root

- Use the outputs from the module to create a subnet in each network; subnet-01 on vpc-01 using CIDR 10.0.1.0/24 and subnet-02 on vpc-02 using CIDR 10.1.1.0/24

10. A proposed solution is at **aws-tf-int/labs/solutions/04**

Step-by-Step Instructions

Set Up the Project Directory

11. Ensure you have moved to the lab directory:

```
cd ~/environment/aws-tf-int/labs/04
```

12. Create subdirectories for the module:

```
mkdir modules
cd modules
mkdir vpc
cd vpc
```

Create the VPC Module

13. Create the following files in **04/modules/vpc**:

main.tf:

```
resource "aws_vpc" "lab_vpc" {
 cidr_block = var.cidr_block
 enable_dns_support = true
 enable_dns_hostnames = true
 tags = {
  Name = var.vpc_name
 }
}
```

variables.tf:

```
variable "vpc_name" {}
variable "cidr_block" {}
```

outputs.tf:

©QA 2026

```
output "vpc_id" {
 value = aws_vpc.lab_vpc.id
}
output "vpc_cidr" {
 value = aws_vpc.lab_vpc.cidr_block
}
```

Create the Root module Configuration

14. Each subnet in AWS must reside in a specific Availability Zone (AZ), which represents an isolated data center within a region. For predictable, repeatable deployments, it is best practice to explicitly specify the availability_zone when creating a subnet. You can use the aws_availability_zones data source to dynamically select valid AZs (e.g. data.aws_availability_zones.available.names[0]) instead of hardcoding AZ names.

15. In **aws-tf-int/labs/04**, create **main.tf**

```
provider "aws" {
 region = "us-west-2"
}

data "aws_availability_zones" "available" {}

module "vpc1" {
 source   = "./modules/vpc"
 vpc_name  = "vpc-01"
 cidr_block = "10.0.0.0/16"
}
module "vpc2" {
 source   = "./modules/vpc"
 vpc_name  = "vpc-02"
 cidr_block = "10.1.0.0/16"
}
resource "aws_subnet" "subnet1" {
 vpc_id    = module.vpc1.vpc_id
```

```
  cidr_block = "10.0.1.0/24"
  availability_zone =
data.aws_availability_zones.available.names[0]
  tags = {
   Name = "subnet-01"
  }
}

resource "aws_subnet" "subnet2" {
  vpc_id     = module.vpc2.vpc_id
  cidr_block = "10.1.1.0/24"
  availability_zone =
data.aws_availability_zones.available.names[1]
  tags = {
   Name = "subnet-02"
  }
}
```

Initialize, Review and Apply

16. From the root module, Initialize terraform and deploy resources...

```
cd ~/environment/aws-tf-int/labs/04
terraform init
terraform plan
terraform apply
```

Verify the Deployment

17. Use **terraform state list** and the Azure Portal to verify the deployment of 2 VPCs in us-east-1, each with a single subnet .

Clean Up Resources

18. Run **terraform destroy --auto-approve**

©QA 2026

Demo Lab

19. This demonstration walks you through a more complex use of modules to deploy VPCs (Virtual Private Clouds) with subnets, Security Groups (SGs), and VPC Peering. The goal is to further highlight the benefits of dynamic and reusable infrastructure by leveraging Terraform modules and outputs.
20. In your IDE, navigate to **/labs/04/demo** and review the provisioned files.

Lab Logic

21. At the top level, the root module defines the AWS provider and statically sets the deployment region (currently us-east-1). It declares a variable called vpcs, which is a map of VPC definitions (for example, vpc1, vpc2). Each entry includes metadata such as the VPC name and its CIDR block.

22. The root module uses a for_each loop to iterate over the vpcs map, deploying each VPC by invoking the reusable VPC module. At the same time, it looks up a separate subnets map — a nested map that specifies which subnets belong to which VPC. This nested map is transformed into a list of subnet objects and passed into the module as input.

23. The root module also contains logic to:

    Create a Security Group for each VPC by looping over the deployed VPCs and calling the SG module, passing the VPC ID from the VPC module's outputs.

    Establish a two-way VPC peering connection between the VPCs, using the vpc_id values output from the VPC modules to wire them together dynamically.

24. This structure ensures that the root module acts purely as an orchestrator:

It passes input data into the child modules.

It consumes and connects outputs from the child modules.

It manages cross-resource dependencies like peering and security relationships cleanly.

25. The VPC module is intentionally reusable. It accepts three inputs:

    vpc_name — the tag and logical name of the VPC.

    cidr_block — the address range for the VPC.

    A list of subnet definitions, where each subnet includes a name and cidr_block.

26. Inside the VPC module:

    A new VPC resource is created with the given CIDR block and tagged with its name.

    The module loops over the list of subnets using for_each to create multiple aws_subnet resources, each linked to the parent VPC and uniquely tagged.

    Note on Availability Zones (AZs): The subnets do not explicitly set an availability_zone, so AWS automatically assigns each subnet to an available AZ within the region at create time. This non-deterministic choice is acceptable for simple setups but can be made explicit if needed for multi-AZ designs.

27. The VPC module exposes two key outputs:

    vpc_details — a map containing the VPC's ID and name, which can be used by other modules (for example, to attach peering or security resources).

    subnets — a map of subnet names to their corresponding subnet IDs, making it easy to attach route tables, instances, or load balancers later.

28. The Security Group module (sg) is separate and reusable. It accepts:

    sg_name — typically based on the parent VPC's name.

    vpc_id — passed in from the vpc module's outputs.

29. The SG module creates a basic security group resource inside the specified VPC and outputs the Security Group ID. Although minimal by default, this module is structured so you can easily also include:

    Ingress and egress rules.

    Tagging standards.

    Environment or application-specific logic.

Deploy the resources

30. Run the following commands:

```
cd ~/environment/aws-tf-int/labs/04/demo
terraform init
terraform plan
terraform apply --auto-approve
```

    Expected output:
    Plan: 9 to add, 0 to change, 0 to destroy.

31. Experiment updating terraform.tfvars, adding and removing subnets in the two VPCs.

32. Destroy all resources before continuing..

```
terraform destroy --auto-approve
```

Congratulations, you have completed this lab

©QA 2026

Copyright
© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

©QA 2026